Distributed Cache Systems

A Deep Dive Into Modern Architecture

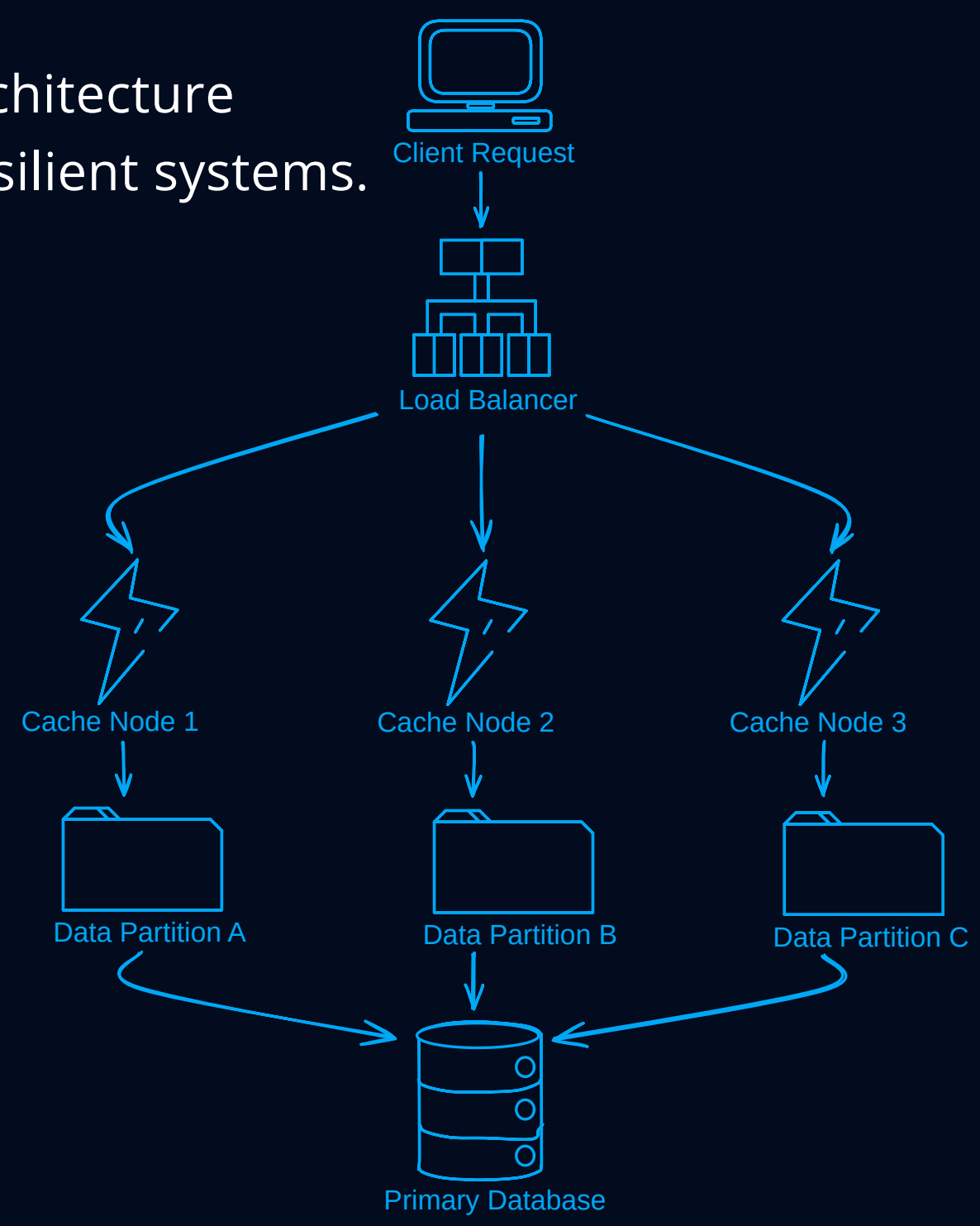
roundz.ai

Why Netflix Never Crashes

Ever wonder how Netflix serves 200+ million users during peak hours without breaking a sweat? Or how Amazon handles Black Friday traffic spikes?

The secret isn't magic—it's **distributed caching**.

Today I'm breaking down the architecture that powers the world's most resilient systems.



roundz.ai

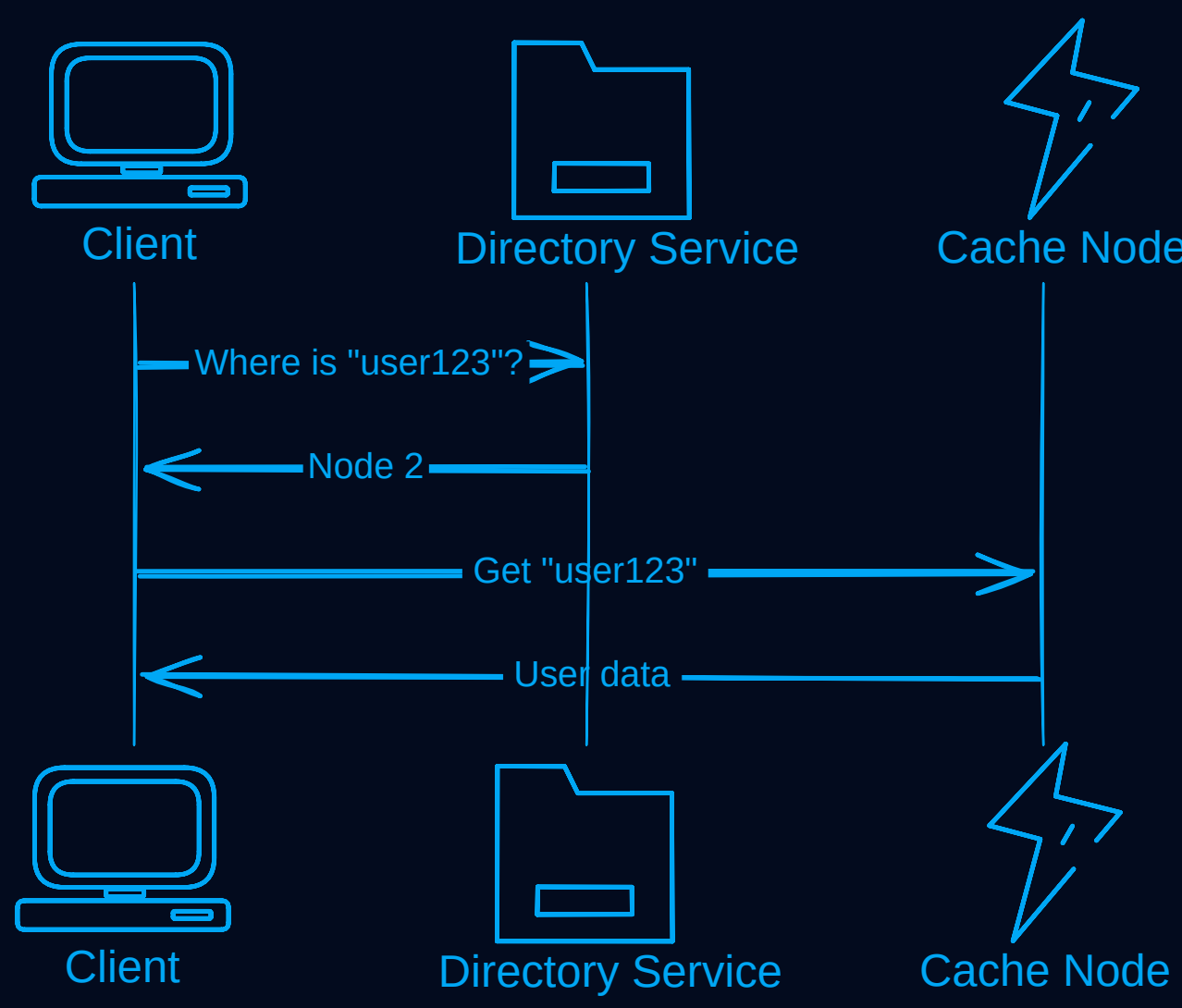
The Impossible Puzzle Every Developer Faces

When building distributed systems, you're trying to achieve four things simultaneously:

- **Fast** (low latency)
- **Available** (always accessible)
- **Consistent** (same data everywhere)
- **Scalable** (handles growth gracefully)

Here's the harsh reality: **You can't have all four perfectly**.

This is where smart trade-offs and architectural decisions make or break your system. The key is knowing which to prioritize for your use case.



roundz.ai

Data Partitioning: The Art of Splitting Without Breaking

Hash-Based Partitioning: Like a smart filing system—run your key through a hash function to know exactly which server handles it.

Consistent Hashing: The game-changer. Think of it as a circular table where adding new members only affects immediate neighbors, not everyone.

Range-Based: Groups related data together (great for geographic distribution).

Directory-Based: A smart coordinator that tracks everything—flexible but can become a bottleneck.

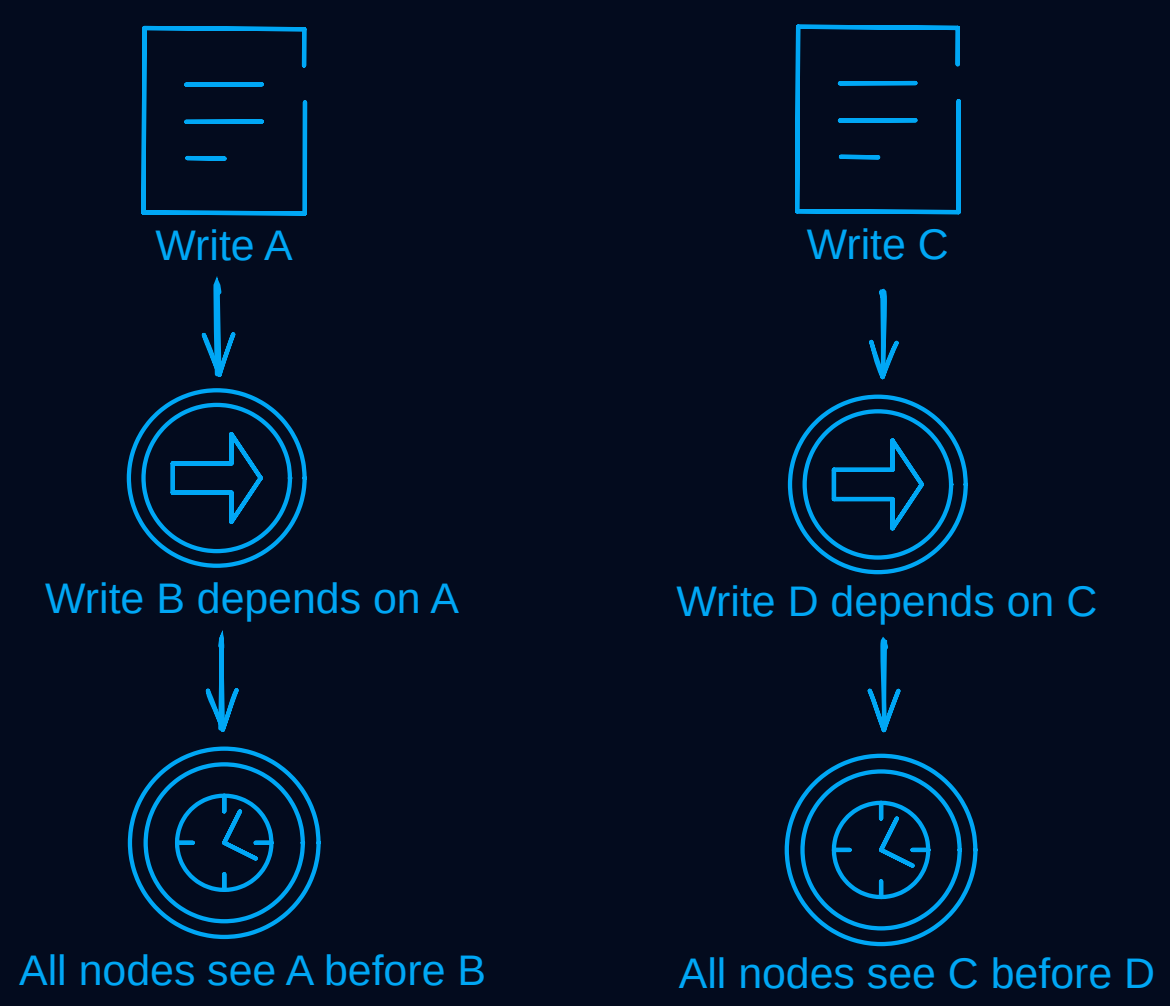
Consistency Models: Choose Your Fighter

Strict Consistency: Every read gets the most recent write. Perfect accuracy but expensive performance-wise.

Eventual Consistency: Data might be temporarily out of sync, but eventually becomes consistent. Powers Amazon DynamoDB and Cassandra.

Causal Consistency: The middle ground—maintains logical ordering where it matters while allowing parallel operations.

The choice depends on your use case: Banking needs strict, social media can handle eventual.



roundz.ai

Cache Eviction: The Bouncer Strategy

Your cache has limited space—you need smart policies about what stays and what goes:

LRU (Least Recently Used): Kicks out data that hasn't been accessed recently. Simple and effective.

LFU (Least Frequently Used): Removes the least popular items based on access frequency.

TLRU (Time-aware LRU): Combines recency with frequency for optimal performance.

Choose based on your access patterns—news sites benefit from LRU, while recommendation engines prefer LFU.

What Can Go Wrong (And How to Fix It)

Cache Stampede: When popular cache expires, million requests hit your database simultaneously.
Solution: Use request coalescing and lock mechanisms.

Hot Partitions: One partition gets overwhelmed while others sit idle.
Solution: Smart load balancing and traffic distribution.

Split Brain: Network partitions cause nodes to disagree on authority.
Solution: Implement proper consensus algorithms.

Prevention is better than cure—design for failure from day one.

Building Your Cache: Start Simple, Scale Smart

Phase 1: Single Redis instance (prototyping)
Phase 2: Master-slave replication (redundancy)
Phase 3: Sharding (horizontal scaling)
Phase 4: Multi-region deployment (global scale)

Key Questions to Ask:
• How consistent does my data need to be?
• What's my read/write ratio?
• Where are my users located?

Monitor everything: Hit rates, latency (p95, p99), memory usage, network traffic.

The Cache Commandments

5 **principles for bulletproof distributed caches**:

- Consistency is a **spectrum**, not binary
- **Measure twice**, **cache once**—know your patterns
- **Plan for failure** (it will happen)
- **Start simple**, evolve gradually
- **Monitor everything** religiously

The best way to learn? Build one yourself, break it, fix it, repeat.

What's your biggest caching challenge? Drop it in the comments—let's solve it together!

Thank You

roundz.ai

Follow us

roundz-ai